ICT INFORMATION AND COMMUNICATIONS TECHNOLOGY ACADEMY

MODULO: Python.5-6

UNITÀ: Py.5-6.1.1

REST & Flask

Introduzione

2024-2026

Introduzione

Rest & Flask

Che cos'è REST

- Architettura *stateless* client-server
- Utilizza HTTP come protocollo di comunicazione
- Principi fondamentali:
  - **Risorse**: identificate da URI
  - **Metodi HTTP**: GET, POST, PUT, DELETE
  - Rappresentazioni: **JSON**, XML
  - **Stateless**: ogni richiesta è indipendente
  - **Cache**: risposte cacheable quando appropriato

REST: Vantaggi

- Semplicità: utilizza standard HTTP esistenti
- Scalabilità: architettura stateless
- Flessibilità: supporta diversi formati dati
- Interoperabilità: funziona su diverse piattaforme
- Separazione: client e server indipendenti

Risorse e URI: progettazione

- Una risorsa = un'entità del dominio (oggetti in analisi concettuale!)
- URI dovrebbero essere nomi (sostantivi), non verbi
- Utilizzare plurali per le collezioni

URI e risorse: esempi corretti

```
GET    /libri          # Ottenere tutti i libri
GET    /libri/5        # Ottenere libro con ID 5
POST   /libri          # Creare nuovo libro
PUT    /libri/5        # Aggiornare libro ID 5
DELETE /libri/5        # Eliminare libro ID 5
```

URI e risorse: esempi scorretti

```
GET    /ottieniLibri

POST   /creaLibro

DELETE /cancellaLibro/5
```

REST: Metodi HTTP e operazioni CRUD

Metodo HTTP	Operazione CRUD	Descrizione	Idempotente?
GET	Read	Recupera risorse	Sì
POST	Create	Crea nuove risorse	No
PUT	Update	Sostituisce completamente	Sì
PATCH	Update	Modifica parziale	No
DELETE	Delete	Elimina risorsa	Sì

Codici di stato HTTP

2xx - Successo:

- `200 OK` - Richiesta completata
- `201 Created` - Risorsa creata
- `204 No Content` - Operazione completata senza contenuto

4xx - Errori Client:

- `400 Bad Request` - Richiesta malformata
- `401 Unauthorized` - Autenticazione richiesta
- `404 Not Found` - Risorsa non trovata
- `409 Conflict` - Conflitto con stato attuale

5xx - Errori Server:

- `500 Internal Server Error` - Errore generico server

JSON come Standard per REST

Vantaggi JSON:

- Leggibile da umani
- Supporto nativo JavaScript
- Parsing efficiente
- Struttura gerarchica

```
{
  "id": 1,
  "titolo": "1984",
  "autore": "George Orwell",
  "isbn": "978-0-452-28423-4",
  "disponibile": true,
  "data_pubblicazione": "1949-06-08",
  "generi": ["dystopian", "political
fiction"]
}
```

Esempio Pratico API REST: Gestione di una libreria

Base URL: https://api.biblioteca.it/v1

```
GET    /libri                    # Lista tutti i libri
GET    /libri?genere=fantasy     # Filtro per genere
GET    /libri/123                # Dettaglio libro
POST   /libri                    # Aggiungi libro
PUT    /libri/123                # Aggiorna libro
DELETE /libri/123                # Elimina libro

GET    /autori                   # Lista autori
GET    /autori/456/libri         # Libri di un autore
```

ICT INFORMATION AND COMMUNICATIONS TECHNOLOGY ACADEMY

MODULO: Python.5-6

UNITÀ: Py.5-6.1.2

REST & Flask

Richardson Maturity Model

2024-2026

API REST: Richardson Maturity Model

REST

Il Richardson Maturity Model

Livelli di Maturità REST

Il Richardson Maturity Model definisce **4 livelli** (0-3) per valutare quanto un'API sia "RESTful":

- **Livello 0**: Un singolo *endpoint* per tutto
- **Livello 1**: Risorse multiple
- **Livello 2**: Verbi HTTP + codici di stato
- **Livello 3**: **HATEOAS** (controlli *ipermedia*)

"True REST" = Livello 3, ma la maggior parte delle API si ferma al Livello 2

Livello 0 - The Swamp of POX

POX = Plain Old XML (o JSON)

Caratteristiche:

- Un singolo endpoint (es. `/api`)
- Sempre `POST`
- Azione specificata nel payload

Problemi: Non sfrutta HTTP, difficile fare cache, poco intuitivo

Esempio:

```
POST /api
Content-Type: application/json
{
  "action": "getBook",
  "bookId": 5
}

POST /api
Content-Type: application/json
{
  "action": "createBook",
  "title": "Nuovo Libro",
  "author": "Autore"
}
```

Livello 1: risorse multiple

```
POST /libri/5            # Ottenere libro
POST /libri              # Operazioni sulla collezione
POST /autori/3           # Operazioni su autore
```

Livello 2: Verbi HTTP + Codici di Stato

```
GET    /libri/5        → 200 OK
POST   /libri          → 201 Created
PUT    /libri/5        → 200 OK
DELETE /libri/5        → 204 No Content
GET    /libri/999      → 404 Not Found
```

La maggior parte delle API REST moderne opera a questo livello

Livello 3 - HATEOAS

HATEOAS = **H**ypermedia **A**s **T**he **E**ngine **O**f **A**pplication **S**tate

Risposta con controlli ipermedia:

Vantaggi:

- Client dinamico e autodescrittivo
- Evoluzione API senza *breaking changes*
- Navigabilità come il web

Svantaggi:

- Maggiore complessità
- Payload più grandi

```
HTTP/1.1 201 Created
Content-Type: application/json
{
  "id": 5,
  "titolo": "1984",
  "autore": "George Orwell",
  "_links": {
    "self": {"href": "/libri/5"},
    "edit": {"href": "/libri/5", "method": "PUT"},
    "delete": {"href": "/libri/5", "method": "DELETE"},
    "author": {"href": "/autori/42"},
    "reviews": {"href": "/libri/5/recensioni"}
  }
}
```